0041 AUTO
0042 APPROVAL
0043 ESCALATE
0044 AUTO
0045 APPROVAL
0046 ASSIST
0047 AUTO
0048 APPROVAL
0049 ESCALATE
0050 AUTO

AI AGENT AUTONOMY GOVERNANCE — MSP & ENTERPRISE IT OPERATIONS

# AutonomyOS

Designing a safe execution layer for enterprise AI agents — the policy layer that decides not what the AI should recommend, but what it should be trusted to do.

> *"AI should earn the right to act."*

`AI recommends › Policy authorizes › Execution follows`

THE FOUR AUTONOMY LEVELS

• AUTO • APPROVAL • ASSIST • ESCALATE

Rohit Mannur
Product Management Portfolio · 2026

01 EXECUTIVE SUMMARY

# More automation, without giving up control

A governance layer for MSP and enterprise IT operations that decides, ticket by ticket and workflow by workflow, whether an AI agent has earned the right to act — or only the right to recommend.

As AI agents become capable of diagnosing and resolving IT support issues, the constraint on adoption stops being *model capability* and becomes *execution authority*: which actions can an AI agent be trusted to carry out on its own, which require a human checkpoint, and which should never be delegated at all.

AutonomyOS is a working prototype of that governance layer. Every incoming ticket moves through the same pipeline — AI diagnosis, knowledge retrieval, deterministic risk scoring, and a fixed policy engine — and lands on exactly one of four outcomes: **AUTO**, **APPROVAL**, **ASSIST**, or **ESCALATE**. The language model proposes; a small set of auditable rules decides.

The product's central bet is that **confidence and permission are different questions**, and that trust should be earned per workflow, evidenced by outcomes, and raised only by an explicit human decision — never by the system itself.

***The product thesis, in one line: AI should earn the right to act.***

ⓘ HOW TO READ THIS DOCUMENT

Every claim in this case study is labeled by its source, so product thinking is never confused with unverified results.

| | |
|---|---|
| VERIFIED | Directly confirmed in the AutonomyOS codebase or README |
| MARKET | Current external research, cited by source |
| HYPOTHESIS | A reasoned product assumption, not yet tested |
| PROPOSED | A future direction, not built in the MVP |

MVP AT A GLANCE — VERIFIED

| | |
|---|---|
| 4 autonomy levels | 1 deterministic policy engine |
| 8 supported action types | 26 backend tests |
| 0 live production integrations — by design | 1 way to raise a ceiling: a human click |

AI REASONING

Diagnoses the ticket, retrieves supporting knowledge, proposes an action, and states a confidence level. Confidence is a belief about the diagnosis — nothing more.

→

EXECUTION AUTHORITY

A fixed, auditable policy evaluates risk, permission, customer impact, reversibility, and track record — and is the only thing that decides whether execution is allowed.

*Next — why execution authority, not diagnostic accuracy, is the real 2026 adoption barrier.*

02 PROBLEM & WHY NOW

# The barrier isn't model capability. It's execution authority.

Organizations are increasingly able to point an AI agent at a support ticket and get a correct diagnosis. That is no longer the hard problem. The hard problem is organizational: **deciding when that agent is allowed to act** on production identity systems, licensing, network infrastructure, and customer-facing accounts.

A wrong recommendation is an inconvenience. A wrong *execution* — a disabled account, an opened firewall port, a granted admin role — is an incident. So the real product question is not "how do we make AI smarter," it is:

## *"How do we safely decide when AI is allowed to act?"*

MARKET WHY THIS IS AN ACTIVE 2026 PROBLEM

**MSP tooling is arriving at the assist layer, not the execution layer.** ConnectWise's 2026 AI Agents and Sidekick lines automate ticket triage, routing and drafting, but by the vendor's own design every suggestion, draft or script still requires a technician's review before anything is sent or executed — the platform is explicit that it is built for assistance first, execution later.

**ITSM platforms are shipping autonomous resolution for narrow, low-risk work.** Freshservice's Freddy AI Agent now resolves requests like password resets and access grants end-to-end, while pairing that autonomy with explicit "govern every action with visibility and control" messaging — automation and governance are being sold as one package, not two.

**The largest platform vendor is building governance as its own product line.** At Knowledge 2026, ServiceNow positioned its AI Control Tower as the governance layer for agents "regardless of where they are built, deployed, or operating" — discovery, approval, and oversight sold separately from the agents that do the work.

Sources: connectwise.com/platform/ai-agents; rallied.ai (Jun 2026); freshworks.com/freshservice/ai-itsm; newsroom.servicenow.com (May 2026); cxtoday.com (May 2026).

CONCEPTUAL LANDSCAPE HYPOTHESIS

Illustrative positioning based on public product messaging, not a verified competitive audit.


FORMAL PROBLEM STATEMENT

Organizations can increasingly deploy AI agents that diagnose and propose fixes for IT support issues. Adoption is gated not by diagnostic accuracy but by the absence of a trusted, auditable way to decide — per action, per workflow — whether that agent should be allowed to execute.

*Next — the single design decision that makes this concrete: confidence versus permission.*

03 THE CORE PRODUCT INSIGHT

# Confidence answers one question. Permission answers another.

AI confidence measures how sure the model is about its own diagnosis. It says nothing about how much damage the proposed action could do if it's wrong. AutonomyOS treats these as two separate axes — and lets the second one override the first.

VERIFIED — DEMO TICKET #4821

M365 authentication failures after a routine password reset. Reversible action, standard permission, low customer impact.

94% AI confidence

DECISION: AUTO

Every safety condition holds at once — low risk, reversible, standard permission, low impact — so high confidence is allowed to translate into execution.

VERIFIED — DEMO TICKET #4823

Production firewall rule change to open an inbound port for a payment gateway integration. Irreversible, privileged, critical blast radius.

88% AI confidence

DECISION: ESCALATE

Risk is CRITICAL. Rule one in the policy engine fires before confidence is even considered: critical blast radius is outside AI execution authority, full stop.

SAME PIPELINE · SAME CONFIDENCE RANGE · OPPOSITE OUTCOME

*“Confidence is a statement about the diagnosis. It is not a statement about the blast radius of the action.”*

A 99%-confident privileged access grant is still a privileged access grant. So the policy engine gates on risk, permission, customer impact, and reversibility *independently* of how sure the model is — confidence can raise a decision toward AUTO, but it can never talk the system past a CRITICAL risk or a privileged permission on its own.

04 PRODUCT OPPORTUNITY & USERS

# A governance layer, not another ITSM platform

AutonomyOS doesn't compete with ticketing systems or diagnostic copilots. It sits between them and the systems they touch — the layer that decides what a diagnosis is allowed to become.


## USERS OF THE GOVERNANCE LAYER

| PERSONA | PRIMARY NEED | PRIMARY FEAR |
|---|---|---|
| **Service Desk Manager**<br>Primary user | Reduce queue volume predictably, with visibility into what's automated and why. | Delegating an action the team can't defend after the fact. |
| **IT / Security Administrator**<br>Primary user | Hard limits on privileged and irreversible actions, independent of AI confidence. | A high-confidence agent talking its way into production access. |
| **Support Engineer**<br>Secondary user | Fast, evidenced recommendations for the tickets AI can't yet execute. | Being asked to rubber-stamp a decision they don't understand. |
| **MSP / IT Leadership**<br>Business stakeholder | Scale service delivery without linear headcount growth. | One bad incident undoing a year of automation gains. |

## JOBS TO BE DONE

FUNCTIONAL

"When AI diagnoses an issue, tell me the safest level of autonomy for that specific action — not a blanket policy for the whole workflow."

EMOTIONAL

"Let me feel that AI on my service desk is controlled, not unpredictable."

BUSINESS

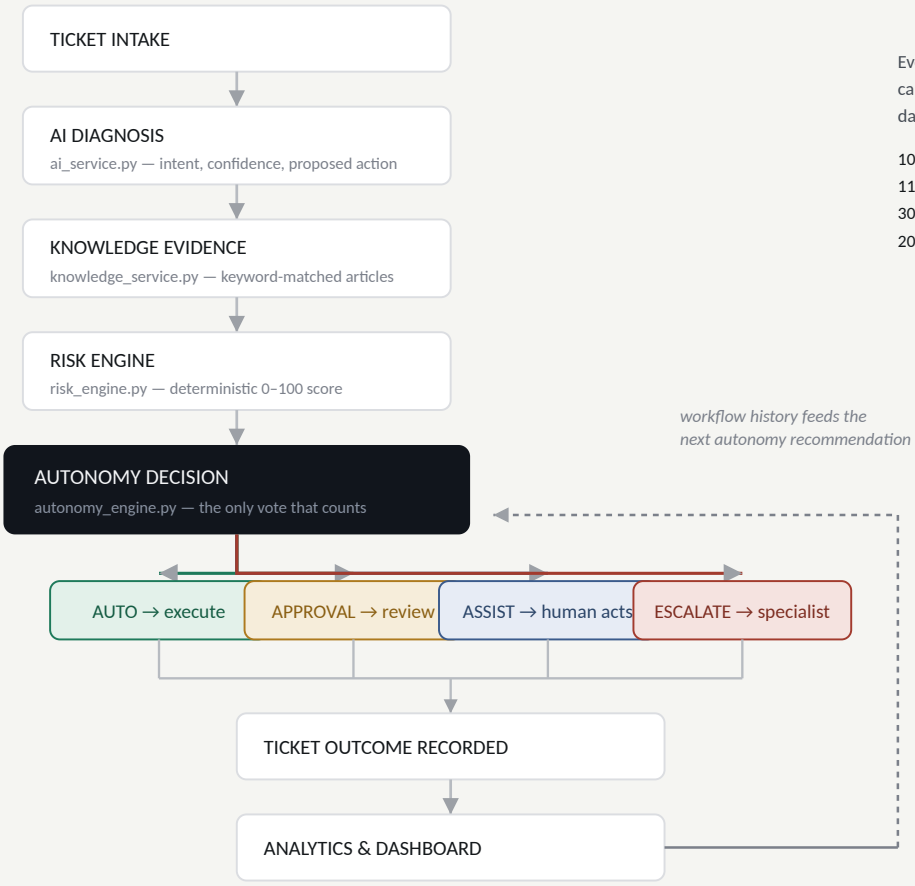
"Help me raise automation without raising the incident rate that comes with it."

*Next — how a single ticket actually moves through the system, end to end.*

05 END-TO-END PRODUCT FLOW VERIFIED

# Every ticket runs the same auditable pipeline

Nothing on the frontend is faked or hardcoded — each stage below is a real API call against a real database, matching the services in `backend/app/services`.




Every stage is a real API call against a live SQLite database. Seed data:

- 10 knowledge articles
- 11 workflow rollups
- 30d trend history
- 20 live demo tickets

### DETERMINISTIC

Same inputs, same decision, every time. The risk score and the autonomy outcome never depend on prompt wording or model version.

### EXPLAINABLE

Every decision returns a `reasons[]` list — the UI shows its work instead of asserting a verdict.

### AUDITABLE

Diagnoses, decisions, executions, and policy changes are all persisted rows, not ephemeral model output.

06 THE AUTONOMY MODEL VERIFIED

# Autonomy is graduated, not a light switch

Instead of a binary AI-or-human split, every ticket lands on one of four levels — the smallest amount of human involvement that risk allows.

**ASSIST**
AI recommends the fix. A human carries it out.

**APPROVAL**
AI prepares the action. A human must approve it first.

**AUTO**
AI executes the action automatically.

+

**ESCALATE**
Not authorized for AI at all — routed to a specialist.

ASSIST → APPROVAL → AUTO is a rising trust spectrum. ESCALATE is a separate off-ramp for anything outside AI's authority — not a step on the same ladder.

## INSIDE THE POLICY ENGINE — `autonomy_engine.py`

Six fixed rules, evaluated in order. The language model supplies confidence and a proposed action; it never touches this logic.

1. **CRITICAL risk → always ESCALATE** — regardless of confidence. This is the rule behind the firewall example on page 4.
2. **Confidence below 70% → ASSIST only** — an unreliable diagnosis doesn't earn a prepared action, only a recommendation.
3. **HIGH risk or PRIVILEGED permission → APPROVAL** — a human gate before execution, no matter how confident the AI is.
4. **Full autonomy (AUTO)** requires every condition at once: confidence ≥ 90%, risk LOW, reversible, STANDARD permission, LOW customer impact.
5. **Earned exception:** MEDIUM risk can still reach AUTO if confidence ≥ 95%, reversible, STANDARD permission, and historical success ≥ 95% — this is literally how a workflow earns autonomy over time.
6. **Default → APPROVAL.** Anything that doesn't clearly clear the AUTO bar goes through a human gate — never straight to execution.

RISK SCORE FORMULA — risk_engine.py

```
score = action_base(LOW 8 / MED 28 / HIGH 50 / CRIT 84)
    + permission_mod(STD 0 / ELEV 6 / PRIV 12)
    + impact_mod(LOW 0 / MED 5 / HIGH 10)
    + irreversible_penalty(10 if not reversible)
```

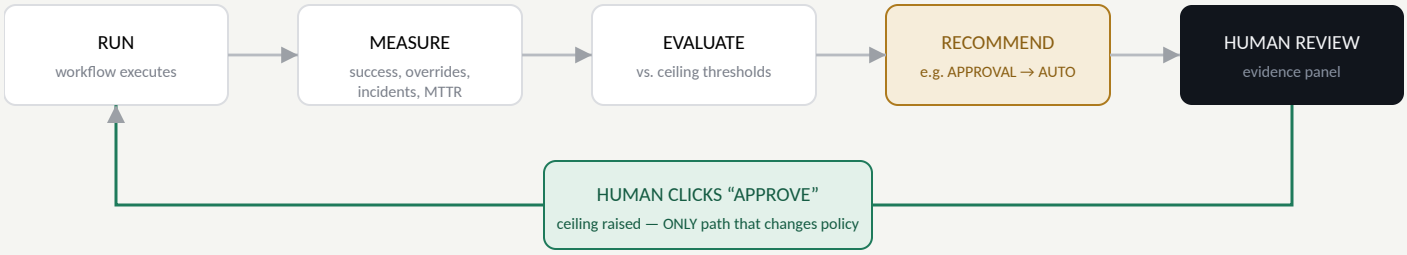
RISK BANDS

```
0–30   LOW
31–60   MEDIUM
61–80   HIGH
81–100 CRITICAL
```

07 TRUST & GOVERNANCE VERIFIED

# Trust is earned per workflow — and never self-granted

AutonomyOS recommends raising a workflow's autonomy ceiling. It never raises one on its own. POST `/api/workflows/{name}/approve-autonomy` is the only code path that changes a ceiling, and it always requires an explicit human click.




## NEVER AUTO-PROMOTED, EVEN WHEN THE DATA LOOKS READY

Workflows that touch privileged or production-critical systems — the codebase names `Privileged Access` and `Firewall Change` specifically — stay human-gated *regardless of track record*. The code comment is explicit: this is a governance decision, not a statistics threshold.

The Analytics screen surfaces a plain-language justification for each candidate: evidence (executions, success rate, override rate, critical incidents, average confidence), the live policy thresholds pulled from the engine itself, and an estimated business impact — illustrated in the product as roughly 43 fewer manual approvals per month for a workflow moving from APPROVAL to AUTO.

### WHY REQUIRE A HUMAN CLICK?

"The data says AUTO is safe" is a statistical claim. "We're comfortable operating with less oversight" is an organizational one. AutonomyOS keeps those two claims separate on purpose — the engine can tell you a workflow looks ready; only a person can decide the organization accepts that trade-off.

*Next — the metric that keeps "automate more" from quietly becoming "automate recklessly."*

08 METRICS FRAMEWORK VERIFIED

# Optimize for safe automation, not raw automation

100% automation is not a win if a meaningful share of it is wrong. The North Star scales raw automation down by how often the workflows currently trusted with AUTO actually succeed.

NORTH STAR METRIC

## Safe Automation Rate

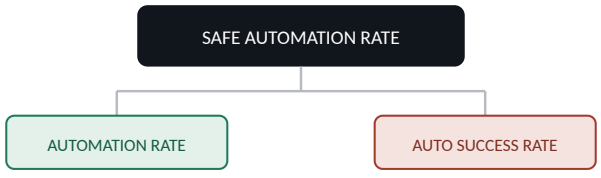
```
safe_automation_rate =
automation_rate × auto_ceiling_success_rate
```

Automation rate scaled by the actual success rate of workflows currently trusted with AUTO. This is mathematically guaranteed to be ≤ the raw automation rate, and it only rises when a workflow earns — and a human approves — a higher ceiling. Never automatically.

GUARDRAILS — PREVENT THE NORTH STAR FROM BEING GAMED

| METRIC | FORMULA | WHY IT MATTERS |
|---|---|---|
| **Incorrect Automation Rate** | `(1 – auto_success_rate) × 100` | Catches automation that is fast but wrong. |
| **Human Override Rate** | `overrides ÷ AI decisions` | Shows where trust hasn't been earned yet. |
| **Escalation Rate** | `escalated ÷ total tickets` | Tracks how often AI correctly recognizes its own limits. |
| **Mean Time to Resolution** | `avg. resolution across workflows` | The efficiency side of the automation/safety trade-off. |

METRICS TREE




guardrails: incorrect automation, override, escalation rate, and MTTR sit under both branches

WHERE THIS COMES FROM

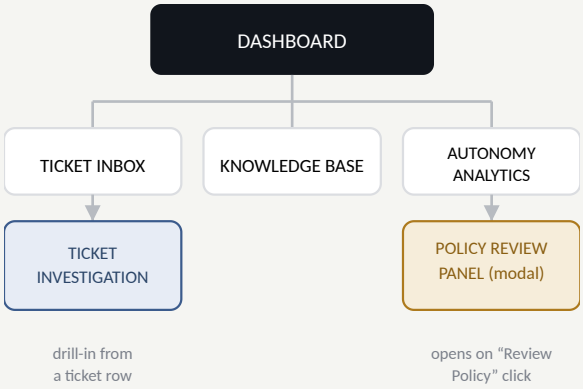
All KPIs are computed server-side in `metrics_service.py` from seeded daily trend and workflow rollup data, and served through `/api/dashboard` and `/api/analytics`. Nothing is hardcoded in the frontend — the dashboard reflects whatever is in the database.

09 PRODUCT EXPERIENCE VERIFIED

# The decision is visible, not just the verdict

Four real screens, matching the app's actual navigation — built so a service manager can see the evidence behind every AUTO, APPROVAL, ASSIST, or ESCALATE.

INFORMATION ARCHITECTURE




Matches the app's actual sidebar: Dashboard, Ticket Inbox, Knowledge Base, Autonomy Analytics.

TICKET INVESTIGATION — conceptual layout, not a literal screenshot

**AI Diagnosis**
Intent · confidence · proposed action · evidence

**Knowledge & Risk**
Matched articles · 0–100 score · breakdown

**Autonomy Decision**
AUTO/APPROVAL/ASSIST/ESCALATE + reasons

A visible AI → Policy → Execution flow diagram sits above the decision card — the moment meant to make the product idea click.

POLICY REVIEW PANEL — conceptual layout, not a literal screenshot

**Evidence**
Executions · success rate · override rate · critical incidents · avg. confidence

**Live Thresholds**
Pulled from the engine, not duplicated in the UI

**Approve AUTO**
(human click)

EXPLAINABILITY AS A FEATURE

Every `AutonomyResult` carries a `reasons[]` list. A service manager can answer "why was this APPROVAL, not AUTO?" without reading code — and can defend the decision to their own leadership.

*Next — the trade-offs behind these choices, and where the prototype knowingly stops.*

10 DECISIONS, RISKS & LIMITATIONS

# Why it works this way — and where it doesn't yet

## PRODUCT TRADE-OFFS

VERIFIED — FROM PROJECT DOCS

**Why not let the LLM decide autonomy?**
Execution authority has to be predictable and auditable. A policy that drifts with prompt wording or model version isn't one an MSP can put its name behind.

**Why not one global autonomy policy?**
A password reset and a firewall change should never share an execution policy — each workflow carries its own risk profile and track record.

**Why human approval instead of a higher confidence bar?**
Confidence describes the diagnosis, not the blast radius. A 99%-confident privileged grant is still a privileged grant.

**Why does a human have to click "Approve AUTO"?**
Raising a ceiling is a governance decision, not a threshold crossing. The engine can say a workflow looks ready; only a person decides the organization accepts it.

## RISK REGISTER

HYPOTHESIS

| RISK | MITIGATION |
|---|---|
| Confidence mistaken for permission | Risk/permission/impact gates evaluated independently of confidence |
| Privileged action executes unsupervised | Rule 3: HIGH risk / PRIVILEGED permission always forces APPROVAL |
| Autonomy ceiling creeps up too fast | Ceiling changes require an explicit human click — never automatic |
| Poor explanation erodes operator trust | Every decision ships a `reasons[]` list in the UI |

## MVP SCOPE

**IN SCOPE**

Diagnosis · evidence · risk scoring · autonomy decision · approvals · simulated execution · escalation · analytics · policy review

**OUT OF SCOPE**

Real M365/AWS/firewall execution · auth · multi-tenancy · vector search · model training · message queue

## LIMITATIONS — STATED PLAINLY

VERIFIED

The AI is a deterministic, keyword-matched mock by default — transparent and reproducible for a demo, not a production NLU system. Execution is fully simulated; no real Microsoft 365, AWS, or firewall integration exists. Knowledge retrieval is keyword-based, not embeddings, which keeps "why did the AI cite this" fully inspectable. Dashboard and Analytics metrics come from seeded historical data, while the 20 interactive tickets are a separate, smaller live demo set. There is no authentication — this is a single-tenant local prototype, and no customer validation is included yet.

11 ROADMAP, VALIDATION & PRODUCT SENSE

# From working prototype to production governance layer

ROADMAP PROPOSED

**Phase 1**
Policy engine, simulated exec, dashboard, governance loop

**Phase 2**
Real integrations: M365, identity, cloud, ticketing

**Phase 3**
RBAC, multi-tenancy, policy builder, approval chains

**Phase 4**
Stronger evidence, calibrated confidence, workflow learning

**Phase 5**
Adaptive, org-specific autonomy policy

## VALIDATION PLAN

PROPOSED — NOT YET CONDUCTED

Interview service desk managers, security admins, and MSP leadership on: what currently blocks AI execution, which ticket types feel safe to automate, what evidence would justify approval, and what incident would be unacceptable. Pair that with quantitative tracking of automation adoption, approval and override rates, and escalation accuracy once real usage exists.

## PRODUCT SENSE DEMONSTRATED

- Reframed “smarter AI” as “safe execution”
- Graduated autonomy over binary automation
- Chose a safety-weighted North Star metric
- Treated explainability as a shipped feature
- Risk-based, not global, policy design
- Trust designed to be earned, not assumed
- Made governance an explicit human action
- Designed escalation as a valid outcome

CLOSING THESIS

# *AI Should Earn the Right to Act.*

AutonomyOS doesn't try to make AI blindly autonomous. It builds a controlled path from capability to execution — evidence, risk, policy, permission — so that more automation never has to mean less control.

AI capability —— evidence —— risk —— policy —— permission —— execution